

Database (SOC-3020)

Alessandro Agostini¹²

aagostini@inha.ac.kr

¹LDKR-Group, School of Computer and Information Engineering
Inha University in Tashkent (IUT), Uzbekistan

²ISGS, Inha University, Incheon, South Korea

WEEK 5



LDKR
Language, Data, Knowledge, Reasoning



DATABASE

SQL: Intermediate

WEEK 5

Code: W5.L1+QUIZ#1

Outline

- 1 From Previous Week
- 2 Modification of the Database
- 3 Exercises
- 4 In-Class Quiz #1 (Lecture 2)

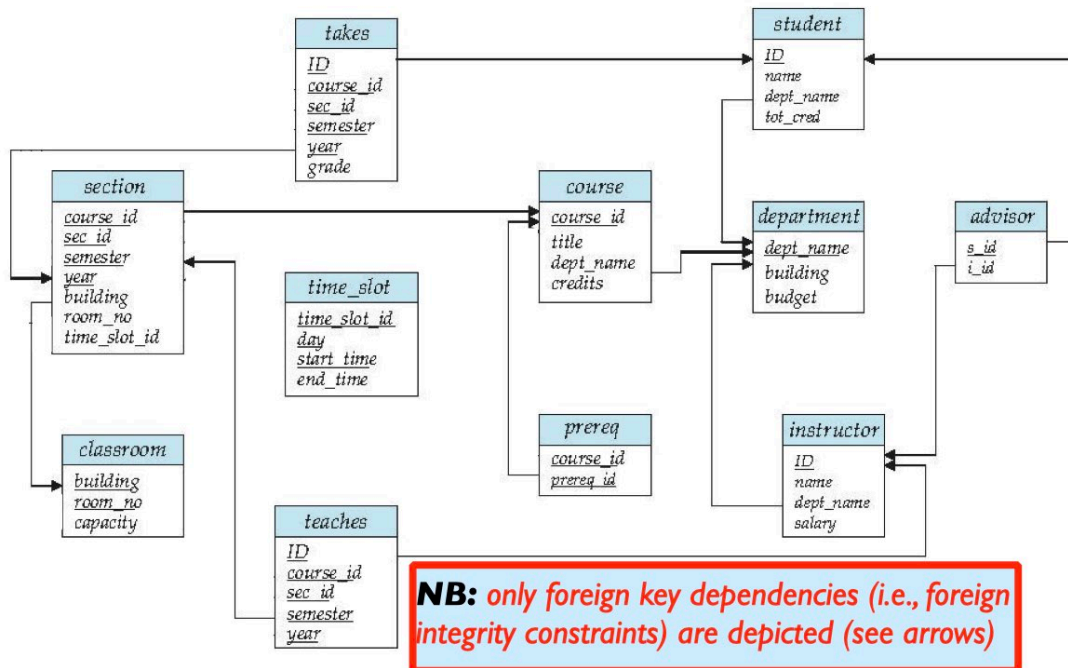


Any question from you?

...



Reminder: Schema for University DB



Reminder: Operations in SQL

SQL
where
select
 [*]
, in from
 [**]
 [**]
 [**]

■ We discussed on the following operations:

- selection of tuples/rows,
 - projection (or: selection of attributes/columns),
 - natural join,
 - Cartesian product,
 - union,
 - set difference,
 - intersection.
- } set operations

■ In blue are “derived” operations, that is, operations defined on other “primary” operations (in red).

[*] new clause **natural join** introduced to be used in **from**

[**] new clauses **union, except, intersect** introduced.





Reminder: Query Structure

- A typical SQL query for data retrieval has the form:

```
select A1, A2, ..., An AggrFunc(B)
from r1, r2, ..., rm
where P is true
group by A1, A2, ..., An
having Q(AggrFunc(B)) true;
```

- A_i attribute used for aggregating (grouping)
- B attribute used to compute the AggrFunc (Aggregate Function)
- r_i input relation (table's schema + instance)
- P logical proposition over **attributes** from r₁, r₂, ..., r_m.
- Q logical proposition using **value aggregated** from AggrFunc

- **where, group by** and **having** are optional.



Reminder: Nested Subqueries

- **Def:** A **subquery** is a **select-from-where** expression (so, a subquery is a SQL query!) that is nested within another query.
- In a SQL query

```
select A1, A2, ..., An
from r1, r2, ..., rm
where P ;
```

- A_i can be replaced by a subquery that generates a **single value**.
- r_i can be replaced by **any valid subquery**.
- P can be replaced with an expression of the form:

Attribute <operation> (**subquery**)

where <operation> is to be defined (see later).

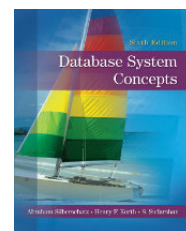


Outline

- 1 From Previous Week
- 2 **Modification of the Database**
- 3 Exercises
- 4 In-Class Quiz #1 (Lecture 2)

Modification of the Database

Ch 3, T; section 3.9





Deletion

- “Delete all instructors”
delete from *instructor*

department
<u>dept_name</u>
building
budget

- “Delete all instructors from the Finance department”

delete from *instructor*
where *dept_name*= ‘Finance’;

<i>instructor</i>
<u>ID</u>
name
dept_name
salary



Deletion: Example

- “Delete all tuples in the *instructor* relation for those instructors associated with a department located in the Watson building”

department
<u>dept_name</u>
building
budget

?

<i>instructor</i>
<u>ID</u>
name
dept_name
salary





Example (cont.)

- “Delete all tuples in the *instructor* relation for those instructors associated with a department located in the Watson building”

department
<u>dept_name</u>
building
budget

delete from *instructor*
where dept_name **in** (**select** dept_name
from *department*
where building = 'Watson');

instructor
<u>ID</u>
name
dept_name
salary



Deletion : An Issue (to be careful about)

- “Delete all instructors whose salary is less than the average salary of instructors”

```
delete from instructor
where salary < (select avg (salary)
               from instructor);
```

I.e., check **where** condition

- **Problem:** as we delete tuples from *instructor*, the average salary changes
- **Solution used in SQL:**
 1. First, compute **avg** (salary) and find all tuples to delete
 2. Next, delete all tuples found above (without recomputing **avg** or retesting the tuples)





Insertion (of plain data)

- “Add a new tuple to *course*”

insert into *course*

values ('CS-437', 'Database Systems', 'Comp. Sci.', 4);

- or equivalently

course's attributes repeated for the
shake of remembering

insert into *course* (*course_id*, *title*, *dept_name*, *credits*)

values ('CS-437', 'Database Systems', 'Comp. Sci.', 4);

- “Add a new tuple to *student* with *tot_creds* set to *null*”

insert into *student* (*ID*, *name*, *dept_name*, *tot_credits*)

values ('3003', 'Green', 'Finance', *null*);



Insertion (of plain data)

- To insert partial data... see SQL manual...
- For instance, *mysql*:

```
INSERT [LOW_PRIORITY | DELAYED | HIGH_PRIORITY] [IGNORE]
[INTO] tbl_name
[(col_name [, col_name] ...)]
{VALUES | VALUE} (value_list) [, (value_list)] ...
```

Source: <https://dev.mysql.com/doc/refman/5.5/en/insert.html>





Insertion (from other table)

student
<u>ID</u>
name
dept_name
tot_cred

- What is the meaning of following query?

```
insert into instructor
select ID, name, dept_name, 20000
from student
where dept_name = 'Comp.Sci' and tot_cred > 145;
```

- Note 1:** “**select**” allows to insert a set of tuples.
- Note 2:** “**select**” must be evaluated before insertions.
(otherwise an *insertion loop* arises)

instructor
<u>ID</u>
name
dept_name
salary

```
insert into student
select *
from student;
```



Updates (to change *some* values)

- “Increase salaries of instructors whose salary is over \$100,000 by 3%, and all others by a 5%”

?





Updates (to change *some* values)

- “Increase salaries of instructors whose salary is over \$100,000 by 3%, and all others by a 5%”
- We can write two **update** statements in sequence:


```
update instructor
set salary = salary * 1.03
where salary > 100000;

update instructor
set salary = salary * 1.05
where salary <= 100000;
```
- The order of above updates is important! (Why?)
- Can be done better using the **case** statement.



“Case” Statement for *Conditional* Updates

- “Increase salaries of instructors whose salary is over \$100,000 by 3%, and all others by a 5%”
- ```
update instructor
set salary = case
 when salary <= 100000 then salary * 1.05
 else salary * 1.03
end;
```





## Updates with Scalar Subqueries

- Recompute and update `tot_creds` value for all students:

*S*: correlation name

**update** student *S*

**set** `tot_cred` = (**select** **sum**(`credits`)

**from** `takes`, `course`

**where** `takes.course_id` = `course.course_id`

**and** `S.ID` = `takes.ID`

**and** `takes.grade` <> 'F'

**and** `takes.grade` **is not null**);

*ID*: correlation variable

| course           |
|------------------|
| <u>course_id</u> |
| title            |
| dept_name        |
| <u>credits</u>   |

| student         |
|-----------------|
| <u>ID</u>       |
| name            |
| dept_name       |
| <u>tot_cred</u> |

| takes            |
|------------------|
| <u>ID</u>        |
| <u>course_id</u> |
| <u>sec_id</u>    |
| <u>semester</u>  |
| <u>year</u>      |
| <u>grade</u>     |

- Update sets `tot_creds` to `null` if `takes.grade` is `null`.



## End of Presentation

Thank you!

# *Questions?*

## Outline

- 1 From Previous Week
- 2 Modification of the Database
- 3 Exercises
- 4 In-Class Quiz #1 (Lecture 2)

# Exercises

Towards In-class Quiz #1

*On the board in class only.*

# *Discussion*

Exercises

## **End of Lecture 1**

Thank you.





## Outline

- 1 From Previous Week
- 2 Modification of the Database
- 3 Exercises
- 4 In-Class Quiz #1 (Lecture 2)

## In-Class Quiz #1

*Only for students who submitted Homework Assignment #1.*  
No slides provided for Quiz #1



## Summary

- 1 From Previous Week
- 2 **Modification of the Database**
- 3 Exercises
- 4 In-Class Quiz #1 (Lecture 2)

**NB:** In **red** are topics in **Video Lecture with mandatory attendance** (eClass).



# **DATABASE**

**End of Slides**

WEEK 5

Code: W5.L1+QUIZ#1